VeriRAG: Efficient Zero-Knowledge Proofs for Verifiable Retrieval-Augmented Generation

Chenqi Lin
linchenqi@pku.edu.cn
Peking University
Beijing, China

Yubo Cui
yuboiverson@gmail.com
Peking University
Beijing, China

Zhelei Zhou
zhouzhelei.zzl@antgroup.com
Ant Group
Beijing, China

Cheng Hong
vince.hc@antgroup.com
Ant Group
Beijing, China

Yufei Wang
DAMO Academy, Alibaba Group
Hangzhou, China
Hupan Lab
Hangzhou, China
wangyufei.wyf@alibaba-inc.com

Zhaohui Chen
DAMO Academy, Alibaba Group
Hangzhou, China
Hupan Lab
Hangzhou, China
chenzhaohui.czh@alibaba-inc.com

Meng Li*
meng.li@pku.edu.cn
Peking University
Beijing, China

Abstract

Retrieval-Augmented Generation (RAG) is widely used to enhance Large Language Models (LLMs), yet the "hallucination" characteristic allows malicious providers to bypass retrieval or claim non-existent data quality. To address these challenges, we present VeriRAG, a framework that leverages Zero-Knowledge Proofs (ZKP) to provide efficient integrity guarantees for RAG systems without compromising dataset privacy. Leveraging the robustness of AI inference, our framework supports Approximate Nearest Neighbor Search (ANNS)-based retrieval to avoid exhaustive searches. For the verification of top-k sorting, we propose an innovative protocol that bypasses the intricate verification of sorting processes. To further enhance performance, we introduce a joint optimization leveraging vector lookup and chunk-merging strategies, which collectively drive down verification overhead while maintaining high generation accuracy. Experimental results demonstrate that VeriRAG scales efficiently to a 37GB dataset, achieving a prover time of 96s and a verifier time of 3s.

CCS Concepts

• **Security and privacy → Cryptography**.

Keywords

Retrieval-Augmented Generation; Verifiable Computation; Large Language Models; Zero-Knowledge Proofs

1 Introduction

Retrieval-Augmented Generation (RAG) [19, 28] has emerged as a cornerstone paradigm for enhancing the capabilities of Large Language Models (LLMs). By integrating relevant external knowledge into the prompt context, RAG enables models to generate more precise, grounded, and context-aware responses. Driven by these advantages, RAG-based systems are now widely deployed in diverse sectors, including healthcare, finance, and legal services [26, 27, 43, 57]. However, the practical deployment of RAG introduces a "trust gap" between service providers and end-users. This tension arises from two primary concerns. First, a provider may claim to utilize high-value proprietary databases—such as premium medical journals or legal archives—while substituting them with low-cost public data to avoid licensing fees. Second, since RAG results in an more than multi-fold increase in prompt length, providers may bypass retrieval to minimize computational overhead, token consumption, and latency. Due to the high linguistic fluency of modern LLMs, outputs generated without retrieval often remain indistinguishable from those that are context-supported. This information asymmetry ensures that users remain unaware of whether they are receiving the high-fidelity service they paid for, highlighting a fundamental challenge in the verifiability of RAG systems.

Zero-Knowledge Proofs (ZKP) [15] offer a promising framework for ensuring the integrity of RAG services. Within this framework, a prover (the service provider) generates a cryptographic proof π to convince a verifier (the user) that the output y is the valid result of a public function f—representing the RAG pipeline—executed on the user's query x and a private witness w. In this context, w typically denotes the knowledge base entries. ZKP guarantees soundness: the verifier will reject the proof with overwhelming probability if the prover deviates from the prescribed computation (e.g., by skipping retrieval). Simultaneously, ZKP maintains zero-knowledge, ensuring that the proof π reveals no information about the witness w beyond its validity. Such privacy is essential for RAG services, as the foundational datasets often contain confidential user data or proprietary intellectual property that must remain shielded from the verifier. (requirement of zk property)

Directly applying a ZKP framework to verify a practical RAG system requires cryptographically proving each step of its underlying

*Corresponding author.

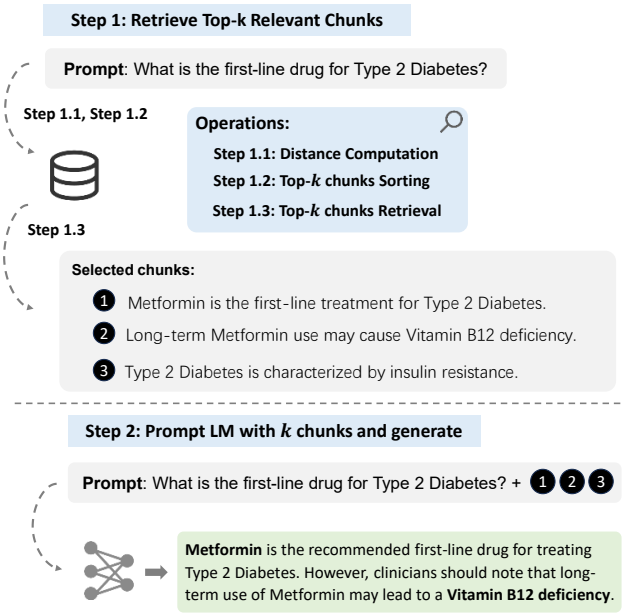

Figure 1: Overview of the standard RAG workflow.

pipeline. As illustrated in Figure 1, the standard RAG workflow computes distances between the query and all N database embeddings, sorts these metrics to identify the top-k candidates, and fetches the corresponding chunks for the Language Model (LM) to generate an augmented response. In plaintext execution, this exhaustive retrieval incurs an $O(N)$ cost for distance evaluation and $O(N \log k)$ for top-k sorting. To mitigate this, Approximate Nearest Neighbor Search (ANNS) techniques, such as Inverted File Product Quantization (IVFPQ), are widely adopted. However, Naively arithmetizing IVFPQ incurs high computational overhead, stemming from three primary structural challenges:

① **ANNS Incompatibility**: Traditional verifiable retrieval schemes typically bind the entire dataset into a single, monolithic cryptographic commitment. IVFPQ, by design, only retrieves and computes over a localized subset of clusters rather than the full database. Efficiently proving that this dynamically accessed subset authentically originates from the global dataset commitment—without falling back to $O(N)$ verification overhead—presents a significant cryptographic challenge.

② **Top-k Sorting Overhead**: Exactly proving a top-k sorting operation poses a severe challenge for ZKP systems due to two primary bottlenecks. First, to enable subsequent conditional data routing and element swapping, the system must evaluate the boolean result of a comparison. Verifying this operation requires bit decomposition [18, 33, 54], which significantly expands the overall circuit size. Second, efficient sorting algorithms, such as min-heaps or quicksort, inherently rely on data-dependent control flow. Dynamically selecting and swapping elements requires accessing data via variable indices, a process that necessitates expensive memory checking protocols [3]. Consequently, the combination of comparison overhead and verifiable dynamic addressing makes standard sorting algorithms inefficient to prove.

③ **Lookup Redundancy**: The bit-length of typical retrieved data chunks (e.g., 4096 bits in our evaluation) significantly exceeds the native scalar field bit-width used in ZKPs (e.g., 254 bits). This physical constraint necessitates decomposing a single chunk into multiple sub-elements. Consequently, fetching a retrieved chunk requires executing redundant lookup arguments across different tables for the exact same index, which severely inflates the constraint system and degrades prover performance.

To overcome these prohibitive bottlenecks, we propose VeriRAG, the first zero-knowledge verifiable framework tailored for practical RAG pipelines. Specifically, our contributions are three-fold:

- We present VeriRAG, the first verifiable framework inherently designed to support ANNS-accelerated retrieval in RAG pipeline. Furthermore, VeriRAG is compatible with zkGPT [42], enabling their integration to provide end-to-end verification for the entire RAG-LLM process.
- We propose a novel top-k verification protocol that significantly reduces the verification overhead. Our protocol requires only a linear number of comparisons and completely eliminates the need to verify expensive data-dependent memory accesses.
- We propose tailored chunk merging strategies leveraging the vector lookup primitive to further optimize system performance. This strategy minimizes the number of processed chunks, resulting in significantly lower prover time for the entire RAG retrieval verification. Extensive evaluations confirm that this optimization is achieved with a negligible impact on generation accuracy.
- We conduct extensive evaluations of VeriRAG on real-world CPU servers. Our results demonstrate that the framework successfully scales to a 37GB dataset, achieving a proof generation time of 96s, a verification time of just 3s, and a compact proof size of 5.21 MB. These results demonstrate VeriRAG's capability to handle large-scale retrieval tasks with minimal verification overhead.

2 Technical Overview

In this paper, we propose VeriRAG, a high-performance verification framework specifically engineered to overcome the computational bottlenecks of ZKPs in RAG pipelines. Recognizing RAG as a critical auxiliary component for LLMs, we strategically align VeriRAG with the state-of-the-art zkGPT [42] architecture. By employing the GKR protocol for linear operations and lookup arguments for non-linear operations, we ensure methodological consistency across the entire LLM-RAG ecosystem.

To address Challenge ①, VeriRAG is systematically architected to support ANNS with a cluster-aware commitment scheme tailored for the IVFPQ algorithm. Instead of binding the entire dataset globally, we construct independent commitments for each cluster. This granular structure enables a highly efficient, verifiable two-stage retrieval pipeline. First, VeriRAG computes the distances between the query vector and all coarse-grained centroids to identify the top-m relevant clusters. Once the verifier authenticates the correctness of these selected cluster indices, the search space is strictly pruned. In the second stage, the prover only needs to open the

specific commitments corresponding to these m clusters to verify the intra-cluster computations and extract the final top-k chunks.

To address Challenge ②, we implement an efficient Top-k verification protocol that avoids the implementation of heavy, data-dependent sorting networks within the circuit. Drawing on techniques from [42, 52], we exploit the asymmetric cost between sorting and verification. Rather than constructing a prohibitive sorting circuit, our approach validates the prover's plaintext top-k results through three operations: a permutation check, a GKR-based difference computation, and a series of range checks. By decoupling the sorting process from the circuit's execution, VeriRAG remains agnostic to the underlying sorting algorithm while maintaining the high efficiency required for real-time RAG applications.

Finally, to address Challenge ③, we employ the vector lookup primitive [12]. In verifiable RAG systems, high-bit-width text chunks must be decomposed into smaller sub-elements to fit within the scalar field. Through this primitive, the integrity of an entire decomposed chunk can be verified via a single vector lookup, preventing a linear increase in proof overhead. Crucially, this cryptographic characteristic reveals a key optimization opportunity: increasing the size of individual data chunks keeps the lookup verification cost nearly constant, yet it significantly reduces the total number of chunks in the database. Based on this insight, we propose two Chunk Merging Strategies: Adaptive Adjacent Merging, based on the narrative continuity of the source context, and Semantic Neighbor Merging, based on global semantic proximity. These strategies aggregate small chunks into larger, contextually cohesive chunks. By significantly reducing the total chunk count, we shrink the search space and accelerate the dominant bottlenecks—distance computation and sorting—throughout the verification pipeline.

3 Related Work

Approximated Nearest Neighbors Search. To balance search efficiency and accuracy in the plaintext domain, existing ANN algorithms generally fall into graph-based [11, 36, 39], tree-based [53], quantization-based[20, 37, 55], and hashing-based[34, 49, 63] categories. While graph-based methods typically yield the best benchmark performance [2] due to their sophisticated vector organization, this structural complexity inherently conflicts with with the strict constraints of advanced cryptographic frameworks, including both secure computation and ZK implementations. As a result, k-means clustering-based algorithms, despite being sub-optimal in plaintext scenarios, have emerged as a more practical and efficient alternative for cryptographic applications.

Zero-Knowledge Proofs for Vector Retrieval significant research has been devoted to ZKP for LLMs [6, 18, 42, 48]; however, these approaches primarily focus on inference and cannot be easily extended to the retrieval stage. Similarly, while many studies have explored ZKP for SQL [29, 52, 61, 62], they are primarily optimized for exact matching and discrete relational operations. Consequently, they fail to accommodate the probabilistic nature of RAG, particularly the non-deterministic ranking inherent in vector-based similarity searches. Recently, a concurrent work [41] also identified this gap and implemented a verifiable IVFPQ scheme using a FRI and Plonk-based architecture. In contrast, our work adopts a Hyrax- and GKR-based architecture. Compared to their approach, our design achieve both a faster prover time and a more compact proof size.

4 Background

4.1 Baseline RAG Workflow

The RAG pipeline typically operates through three integrated stages: **indexing**, **retrieval**, and **generation**. During **indexing**, a corpus of raw data is partitioned into semantically coherent "chunks" and transformed into high-dimensional vectors via an embedding model for storage in a vector database. Upon receiving a user query, the retrieval stage first encodes the input into a query vector using the identical embedding model. It then performs a search against the indexed database using standard distance metrics (e.g., L_2 distance or cosine similarity) to identify the top-k nearest document chunks. Finally, in the **generation** phase, these chunks are concatenated with the original query to form an augmented prompt for the LLM.

The workflow of RAG can be divided into three distinct phases with different verification requirements. The indexing phase is typically performed by the server in an offline manner, and thus is beyond the necessity of real-time verification. For the online phases, existing frameworks have provided solutions to verify the integrity of LLM inference, which covers both the embedding and generation stages. However, the retrieval phase remains a significant blank in current ZKP research.

4.2 Approximate Nearest Neighbor Search

In the retrieval phase, RAG must identify the top-k most similar document chunks to a query within a massive vector space—a process fundamentally defined as a Nearest Neighbor (NN) search. While exact NN search ensures maximum precision, it is computationally expensive and introduces significant latency in large-scale deployments. Given that many RAG applications can tolerate search inaccuracy in exchange for higher throughput, Approximate Nearest Neighbor (ANN) search has become the de facto standard. Among various ANN techniques [11, 20, 34, 36, 37, 39, 49, 53, 55, 63], Inverted File Index with Product Quantization (IVFPQ)[20] is particularly prominent due to its balance of speed and memory efficiency. It serves as the backbone for top-performing ANN frameworks such as FAISS[24] and ScANN[16], and has been integrated into recent RAG-optimized systems such as PipeRAG[23], Chameleon[22], and AiSAQ[50].

The IVFPQ workflow is fundamentally structured into two distinct phases: Offline Indexing and Online Retrieval. The offline phase prepares the dataset through two primary operations: Inverted File Indexing (IVF) and Product Quantization (PQ). During Online Retrieval, the system processes a query through three stages: Coarse Cluster Filtering, PQ-based Fine-grained Ranking, and Top-k Chunk Retrieval. An illustrative example of this workflow is provided in Figure 2. Within this framework, the similarity between a query vector q and a database vector x is typically quantified using the squared Euclidean distance (L_2), formulated as:

$$L_2(q, x) = \sum_{i=0}^{D-1} (x_i - q_i)^2$$

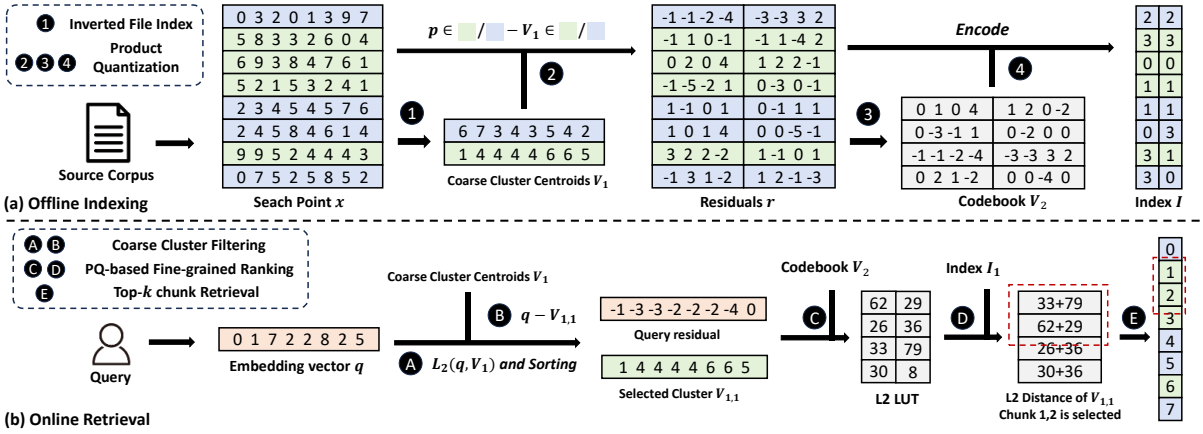

Figure 2: The example of (a) offline Indexing and (b) online retrieval component of IVFPQ-based ANN search.

This metric serves as the computational basis for both identifying the closest coarse clusters and estimating distances between the query and the quantized data points.

Then we will decrible the workflow in detail:

4.2.1 Offline Indexing. The offline indexing transforms raw high-dimensional vectors into a searchable structure through two primary operations: Inverted File Indexing and Product Quantization.

Inverted File Index (IVF): ❶ During this process, the dataset of search points x is partitioned into N_c distinct clusters to narrow the search space. These clusters are defined by a set of centroids, typically pre-calculated through algorithms such as K-means clustering. As illustrated in our example, a set of eight search points is organized into two clusters ($N_c = 2$).

Product Quantization (PQ): The PQ method is widely used for vector compression. ❷ The process begins by decomposing the D-dimensional vector space into $m = D/d$ disjoint subspaces, each with a dimensionality of d. Following our example, an 8-dimensional search point ($D = 8$) is partitioned into two sub-vectors ($m = 2$), each of length 4 ($d = 4$). ❸ PQ is typically applied to the residuals—the vector differences between the original search points and their assigned "first-level" IVF centroids. Within each subspace, these residuals are clustered into E "second-level" clusters (e.g., $E = 4$ in our case) using K-means. The resulting centroids form a codebook, where each centroid is assigned a unique index. ❹ During the encoding phase, each sub-vector is replaced by the index of its nearest centroid in the corresponding codebook.

4.2.2 Online Retrieval. Given a user query, the online retrieval system traverses the pre-computed index through a three-stage pipeline—Coarse Cluster Filtering, PQ-based Fine-grained Ranking, and Top-k Chunk Retrieval—to execute the ANN search.

Coarse Cluster Filtering: Ⓐ The system first computes the pairwise L_2 distances between the query vector and the N_c coarse centroids in the IVF index. The system selects the m closest centroids and restricts all subsequent search operations exclusively to the points within those corresponding clusters. our example adopts a selective approach where $m = 1$. Ⓑ Next, the query calculates residuals between the query embedding and each selected centroid.

Product Quantization (PQ)-based Fine-grained Ranking: Ⓒ For each of these residuals, the system calculates the pairwise L_2 distances between its D/M sub-vectors and every corresponding entry in the E codebooks. These pre-calculated values are organized into an L2-Lookup-Table (L2-LUT) to facilitate rapid distance estimation. Ⓓ Then, the system performs a rapid scan of all encoded search points within the m selected clusters. For each encoded point, the total distance to the query is approximated by aggregating the pre-computed values from the L2-LUT.

Top-k Chunk Retrieval: Ⓔ Based on the computed distance scores, the system identifies and returns the top-k nearest neighbors. While this step is computationally trivial in a plaintext setting, verifying that the selected chunks indeed belong to the pre-determined clusters requires a customized verification protocol.

4.3 Foundations of Arithmetic Operation Verification

The GKR protocol has been extensively adopted in the field of ZKP due to its computational efficiency. Leveraging the sumcheck protocol [35] and multilinear extensions (MLE) [8] as core building blocks, the GKR protocol [14] provides a robust framework for verifying the integrity of arithmetic circuit evaluations.

4.3.1 Sumckeck protocol. The sumcheck protocol is a cornerstone of interactive proofs, designed to verify the sum of an ℓ-variate polynomial f over the Boolean hypercube: $H = \sum_{b \in \{0,1\}^\ell} f(b)$. Instead of an exponential number of evaluations (2^ℓ), the protocol utilizes an ℓ-round interaction to reduce the claim about the total sum to a single evaluation of f at a random point $(r_1, \ldots, r_\ell) \in \mathbb{F}^\ell$. In each round, the prover provides a univariate polynomial representing a partial sum, which the verifier checks for consistency against the previous round's challenge. The protocol concludes with a single oracle query to f, effectively shifting the heavy summation task to the prover while maintaining a linear communication complexity relative to the number of variables.

4.3.2 Multilinear Extension.

Definition 1 (Identity Function). Let $\ell \in \mathbb{N}$. The identity function $eq : \{0,1\}^\ell \times \{0,1\}^\ell \to \{0,1\}$ is defined such that $eq(x,y) = 1$ if $x = y$, and $eq(x,y) = 0$ otherwise.

Definition 2 (Multilinear Extension). Let $V : \{0,1\}^\ell \to \mathbb{F}$ be a function. The *multilinear extension* of V is the unique polynomial $\tilde{V} : \mathbb{F}^\ell \to \mathbb{F}$ such that $\tilde{V}(x) = V(x)$ for all $x \in \{0,1\}^\ell$. The polynomial $\tilde{V}$ can be evaluated as: $\tilde{V}(x_1, \ldots, x_\ell) = \sum_{b \in \{0,1\}^\ell} \tilde{eq}(x,b) \cdot V(b)$, where $\tilde{eq}(x,b) = \prod_{i=1}^{\ell}(x_i b_i + (1-x_i)(1-b_i))$.

While the sum-check protocol is designed to prove the sum of a polynomial over the Boolean hypercube, practical applications often involve data represented as vectors. Multilinear Extension effectively bridges this gap. Specifically, for an array $a = (a_0, \ldots, a_{N-1})$ where N is a power of 2, we view it as a function $a : \{0,1\}^{\log N} \to \mathbb{F}$ such that $a(i_1, \ldots, i_{\log N}) = a_i$ for each index i. Consequently, we extend the notion of multilinear extension to arrays, denoting $\tilde{a}$ as the MLE of the function represented by array a.

4.3.3 GKR protocol. The GKR protocol employs the sumcheck protocol and Multilinear Extension to verify circuits composed of addition and multiplication gates. Traditionally, each gate processes at most two inputs from the preceding layer. [33] generalizes this structure, allowing gates to receive inputs either from the previous layer or directly from the input layer, while supporting multi-input additions and sums of multiple products.

In line with the notation in [33], let $V_i : \{0,1\}^{s_i} \to \mathbb{F}$ represent the outputs of layer i, where V_{in} denote the input layers, respectively. We let $\tilde{V}_i$ be the MLE of V_i and $\tilde{V}_{i,in}$ be the MLE of the input subset connected to layer i. At each layer i, the prover and verifier use the sum-check protocol to reduce evaluations of $\tilde{V}_i$ to those of $\tilde{V}_{i-1}$ and $\tilde{V}_{i,in}$. Finally, the evaluations of $\tilde{V}_{i,in}$ collected across all layers are combined into a single evaluation of the global input MLE, $\tilde{V}_{in}$, via an additional sum-check.

4.4 Foundations of Non-Arithmetic Operation Verification

4.4.1 Lookup Argument. Lookup protocols are essential building blocks in ZKP for verifying non-arithmetic operations. Prior research has primarily leveraged these protocols for range checks—for instance, verifying that a value x falls within the set $\{0, 1, \ldots, 2^k - 1\}$ to prove it is a k-bit non-negative integer. Such applications typically require only an unindexed lookup over a static public table. Formally, an unindexed lookup ensures set membership: $\forall i \in [0, m), f_i \in t$, where $f \in \mathbb{F}^m$ is the witness vector and $t \in \mathbb{F}^n$ is the public table. Following the approach in [42, 48], we utilize LogUp[17, 38], which reduces the lookup relation to the following rational identity:

$$\sum_{i=0}^{m-1} \frac{1}{\gamma + f_i} = \sum_{j=0}^{n-1} \frac{c_j}{\gamma + t_j}$$

Here, $\gamma \in \mathbb{F}$ is a random challenge provided by the verifier, and $c \in \mathbb{F}^n$ is a multiplicity vector where each c_j represents the frequency of the table entry t_j within the witness vector f. This identity is subsequently verified with high efficiency using the sumcheck protocol.

Verifying the RAG process introduces sophisticated requirements, such as sorting and chunk retrieval, which necessitate *indexed lookup* and *indexed permutation* protocols. Formally, an indexed lookup enforces $f_i = t_{a_i}$ for a provided index vector a, while an indexed permutation is its special case ensuring $f_i = t_{\sigma(i)}$ for a known permutation σ. As demonstrated in recent works [5, 47], standard unindexed lookups can be naturally extended to support these indexed variants by integrating position information into the values via a random challenge. This allows us to satisfy the complex retrieval requirements of RAG efficiently.

4.4.2 Vector Lookup Primitive. A powerful property of modern lookup arguments is their ability to efficiently verify vector tuples via Random Linear Combinations (RLC). Specifically, multiple lookup instances $\{f_i \subset T_i\}_{i=1}^w$ sharing a common access index can be compressed into a single lookup. By utilizing a random challenge $\alpha \in \mathbb{F}$ from the verifier, the individual tables and witnesses are aggregated as $T^* = \sum_{i=1}^w \alpha^i T_i$ and $f^* = \sum_{i=1}^w \alpha^i f_i$. Consequently, the verification is reduced to a single lookup relation asserting $f^* \subset T^*$. This effectively verifies all w sub-elements simultaneously within a single protocol instance, circumventing the need for w separate lookups. We refer to [13] for the formal completeness and soundness analysis.

4.5 Zero-Knowledge Arguments and Commitments

Zero-Knowledge Arguments. A zero-knowledge argument allows a computationally bounded prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that a computation $y = C(x, w)$ is executed correctly on a public instance x and a private witness w. Such protocols inherently guarantee three properties: *completeness* (honest provers are accepted), *soundness* (malicious provers are rejected), and *zero-knowledge* (the witness w remains entirely hidden). We defer the rigorous mathematical formulations of these properties to Appendix B.

Zero-Knowledge Polynomial Commitments (zkPC) and GKR. Following established paradigms [51, 56, 60, 61], standard GKR protocols can be lifted to zero-knowledge arguments via zkPC schemes. Specifically, in the final GKR round, the verifier requires the multilinear extension (MLE) of the circuit's input evaluated at random points. To protect the secret witness, $\mathcal{P}$ commits to its MLE using a zkPC and subsequently opens the required evaluations securely. We adopt this exact framework to construct our VeriRAG, and provide the formal definitions of zkPC in Appendix C.

5 VeriRAG Framework

5.1 Definition

We present VeriRAG as an interactive protocol between a cloud server (prover) and a client (verifier). For each retrieval request, the prover constructs a succinct zero-knowledge proof based on the public statement $\mathbf{q}$ (the user query) and a composite secret witness $(\mathrm{w}_{db}, \mathbf{c})$. Here, w_{db} denotes the underlying database state (including all cluster centers, codebooks, data chunks, etc.), while $\mathbf{c}$ denotes the secretly retrieved data chunk. To protect dataset privacy, $\mathbf{c}$ remains hidden and is directly fed into the subsequent LM generation phase. If both the retrieval and generation proofs pass verification, the client attains a guarantee that the final model

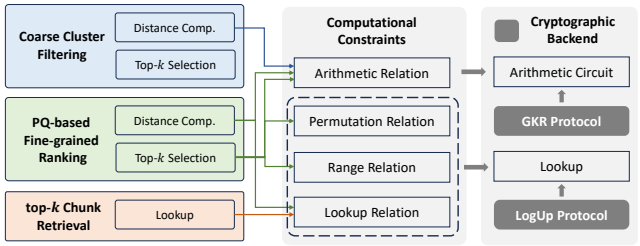

Figure 3: Overview of the prover workflow, mapping the IVFPQ-based retrieval stages to their corresponding zero-knowledge constraints and proof systems.

output is faithfully derived from the most relevant chunks within the committed database. Abstractly, VeriRAG provides an efficient proof system of retrieval for the following instance-witness relation:

$$\mathcal{R}_{\text{VeriRAG}} := \left\{ \left(\mathbf{q} \; ; \; (\mathsf{w}_{db}, \mathbf{c})\right) \; : \; \text{IVFPQ}(\mathbf{q}, \mathsf{w}_{db}) = \mathbf{c} \right\} \tag{1}$$

Here, the abstract IVFPQ function can be decomposed into a combination of arithmetic and non-arithmetic relations to be proven based on Figure 3. IVFPQ is a complex function, which consists of three phases. In the first two phases, distance computations primarily yield arithmetic relations. Furthermore, the verification of top-k selection in these phases necessitates the use of permutation and range-check relations alongside the arithmetic ones. Finally, verifying the Top-k chunk retrieval is entirely reduced to a set of lookup relations.

Following [9, 33], we present the definition of our VeriRAG system. More precisely, a VeriRAG system should consist of the following algorithms:

- $\mathsf{pp} \leftarrow \text{VeriRAG.KeyGen}(1^\lambda)$: Given the security parameter λ, this algorithm generates the public parameters pp.
- $\mathsf{com}_{db} \leftarrow \text{VeriRAG.Commit}(\mathsf{w}_{db}, \mathsf{pp}, r_{\mathsf{w}_{db}})$: The algorithm commits to the entire database state w_{db} using randomness r, producing a commitment com_{db}. This is typically executed once during an offline preprocessing phase.
- $\pi \leftarrow \text{VeriRAG.Prove}(\mathsf{w}_{db}, \mathbf{q}, \mathbf{c}, \mathsf{pp}, r_{\mathsf{w}_{db}})$: Given the public query $\mathbf{q}$ and the composite secret witness $(\mathsf{w}_{db}, \mathbf{c})$, the prover evaluates the retrieval algorithm and generates a proof π. This proof attests that $\mathbf{c}$ is faithfully retrieved from the database without revealing w_{db} or $\mathbf{c}$.
- $\{0,1\} \leftarrow \text{VeriRAG.Verify}(\mathsf{com}_{db}, \mathbf{q}, \pi, \mathsf{pp})$: The verifier takes the database commitment com_{db}, the public query $\mathbf{q}$, and the proof π as inputs. It outputs 1 (accept) if the proof is valid, and 0 (reject) otherwise.

The VeriRAG algorithms constitute a zk-SNARK for retrieval. Specifically, the scheme satisfies *completeness*, ensuring that an honest prover executing a valid retrieval process will always successfully convince the verifier. It also achieves *soundness*, meaning the probability that a malicious prover fabricates a proof for an invalid retrieval process and successfully passes verification is strictly negligible. Furthermore, it guarantees *zero-knowledge*, ensuring that the proof π leaks absolutely no information regarding the prover's secret database w_{db} or the exact content of the retrieved chunk $\mathbf{c}$.

Formally, the proposed VeriRAG algorithms constitute a zk-SNARK for the retrieval relation $\mathcal{R}_{\text{VeriRAG}}$. Specifically, the scheme satisfies *completeness*, ensuring that an honest prover executing a valid retrieval process will always successfully convince the verifier. It also achieves *soundness*, meaning the probability that a malicious prover fabricates a proof for an invalid retrieval process and successfully passes verification is strictly negligible. Furthermore, it guarantees *zero-knowledge*, ensuring that the proof π leaks absolutely no information regarding the prover's secret database w_{db} or the exact content of the retrieved chunk $\mathbf{c}$.

5.2 Protocol Details

To seamlessly integrate the retrieval proof with the ensuing Transformer computations (i.e., embedding and generation), we adopt the consistent proof framework of [42]. Following this approach, the arithmetic relations are represented as arithmetic circuits and proven via the GKR protocol. Conversely, the non-arithmetic relations—encompassing range checks, permutations, and table lookups—are efficiently handled using the LogUp protocol. Figure 4 illustrates the overall VeriRAG protocol, and we detail its core components and the workflow between the prover ($\mathcal{P}$) and verifier ($\mathcal{V}$) below.

5.2.1 Offline Phase. Conceptually, VeriRAG treats the outputs of the offline indexing phase—including document chunking, embedding extraction, and and the offline construction of the IVFPQ index—as a fixed "global state," analogous to pre-trained model weights in verifiable machine learning. We do not generate proofs for the data preprocessing or embedding extraction itself. Instead, the prover $\mathcal{P}$ merely publishes commitments of these outputs.

Specifically, during this offline phase, the $\mathcal{P}$ executes a two-stage process comprising coarse clustering and Product Quantization. This process yields the coarse centroids $\mathbf{V}_1$ and the PQ sub-centroids $\mathbf{V}_2$, alongside the quantized codes $\mathbf{I}_i$ and corresponding data chunks $\mathbf{C}_i$ for each cluster i. The $\mathcal{P}$ then generates commitments for the global parameters $\mathbf{V}_1$ and $\mathbf{V}_2$. Because these components are static, this preprocessing is a one-time operation, allowing its computational cost to be amortized across all future user queries. To maintain ZK efficiency, $\mathcal{P}$ commits to the database on a per-cluster basis rather than globally. This granularity allows the subsequent GKR and lookup arguments to evaluate only the relevant clusters, directly avoiding a full database scan.

5.2.2 Phase I: Coarse Cluster Filtering. Given the query $\mathbf{q}$, $\mathcal{P}$ identifies the top-m coarse centroids. To enable verification, $\mathcal{P}$ computes several query-dependent intermediate witnesses: the distance vector $\mathbf{d}$ between $\mathbf{q}$ and centroids $\mathbf{V}_1$, its permuted version $\mathbf{d}_\text{p}$, lookup queries $\mathbf{l}$, and the residual vector $\mathbf{r}$ (capturing differences between $\mathbf{q}$ and the selected centroids). $\mathcal{P}$ commits to these witnesses and sends them to $\mathcal{V}$ alongside the permutation map σ. Note that these commitments must be dynamically generated per request, a paradigm that similarly applies to the subsequent computations in Phases II and III.

The verification proceeds in two stages. First, to verify the distance computation, we rely on the pre-normalization of the query $\mathbf{q}$ and each coarse centroid in $\mathbf{V}_1 = [\mathbf{v}_1, \ldots, \mathbf{v}_{n_c}]^\top$ (i.e., $\|\mathbf{q}\|_2 = 1$ and $\|\mathbf{v}_i\|_2 = 1$). This rigorously simplifies the squared Euclidean distance to $\|\mathbf{q} - \mathbf{v}_i\|_2^2 = 2 - 2\langle \mathbf{q}, \mathbf{v}_i \rangle$, allowing $\mathcal{P}$ and $\mathcal{V}$ to efficiently

prove the distance vector $\mathbf{d}$ via a GKR protocol over these inner products. Second, to verify the top-m cluster selection, they execute our proposed protocol integrating a permutation check, a GKR-based difference check, and a range check. Finally, because the permutation map becomes public after this selection, a single appended GKR layer suffices to ensure the correctness of the residuals $\mathbf{r}_j = \mathbf{q} - \mathbf{v}_{\text{selected},j}$ for $j \in [1, m]$.

In this step, the permutation mapping σ is made public. This approach safely preserves data privacy because while σ reveals coarse-grained access patterns (i.e., which clusters are selected), it does not leak the specific semantic contents within them. Furthermore, by making σ public, $\mathcal{V}$ can independently fetch the corresponding cluster commitments for the next phase, eliminating the overhead of verifying extra data-dependent accesses.

5.2.3 Phase II: Fine-grained Ranking. Following coarse selection, $\mathcal{P}$ applies Product Quantization (PQ) to identify the top-k nearest vectors within the candidate clusters. $\mathcal{P}$ first constructs distance lookup tables $\mathbf{T}$ containing the partial distances between the residuals $\mathbf{r}$ and sub-centroids $\mathbf{V}_2$. By retrieving sub-distances via the quantized codes $\mathbf{I}_i$, $\mathcal{P}$ aggregates them into the final PQ distances $\mathbf{d}'$ for each candidate. For verifiable top-k selection, $\mathcal{P}$ generates the permuted distance vector $\mathbf{d}'_{\text{p}}$ and associated lookup queries $\mathbf{l}'$. $\mathcal{P}$ then commits to these intermediate witnesses and sends them to $\mathcal{V}$ alongside the public permutation map σ'.

The verification process then transitions to the fine-grained ranking phase, which consists of the validation of aggregated PQ distances computation and the final selection of the top-k data chunks. Note that the top-k selection follows a verification protocol identical to the one used for the Phase I. The computation of $\mathbf{d}'$ is verified through a three-step process:

① **Lookup Table Construction:** First, $\mathcal{P}$ and $\mathcal{V}$ execute a GKR protocol to prove the correct generation of the lookup tables $\mathbf{T}$, which store the distances between the residual vector $\mathbf{r}$ and the sub-centroids $\mathbf{V}_2$; ② **Sub-distance Retrieval:** Second, using the PQ codes $\{\mathbf{I}_i\}_{i=1}^{m}$ of the selected clusters as indices, the parties utilize the Logup protocol to prove that the sub-distances $\mathbf{d}_{\text{codes}}$ are faithfully retrieved from $\mathbf{T}$. Crucially, since $\mathcal{V}$ already possesses the permutation σ from the previous phase, it can selectively aggregate the relevant commitments $\{com_{\mathbf{I}_{\sigma[i]}}\}_{i=0}^{m-1}$ to ensure that the indices remain consistent with the coarse clustering results; ③ **Distance Aggregation:** Finally, an additional GKR layer is executed to verify that each entry in $\mathbf{d}'$ is the correct summation of its constituent sub-distances.

5.2.4 Phase III: Top-k Chunk Retrieval. After determining the indices of the selected chunks, $\mathcal{P}$ constructs a concatenated lookup table $\mathbf{T}_c = \mathbf{C}_{\sigma[0]} \parallel \mathbf{C}_{\sigma[1]} \parallel \cdots \parallel \mathbf{C}_{\sigma[m-1]}$. Utilizing the previously revealed public permutations σ and σ', $\mathcal{P}$ retrieves the target data chunks, denoted as $\mathbf{f}$. Finally, to ensure the integrity of the retrieval, $\mathcal{P}$ and $\mathcal{V}$ execute the Lookup protocol, proving that each entry $\mathbf{f}[i]$ corresponds correctly to $\mathbf{T}_c[\sigma'[i]]$ for all $i \in \{0, \ldots, k-1\}$.

5.3 Verifiable Top-k Identification

Inspired by the maximum/minimum verification techniques in [52] and non-linear operation handling in [42], our approach leverages the principle of asymmetric cost. While sorting N elements is computationally intensive, verifying the correctness of a Top-k result can be achieved with significantly lower complexity. To bypass the high overhead of in-circuit sorting, our protocol decomposes Top-k verification into a coordinated sequence of permutation checks, GKR-based arithmetic, and range proofs:

(1) **Permutation Check:** $\mathcal{P}$ and $\mathcal{V}$ utilize the Lookup protocol to ensure that the reordered distance vector $\mathbf{d}_{\text{p}}$ is a valid permutation of the original distance vector $\mathbf{d}$. Specifically, $\mathcal{P}$ proves that: $\mathbf{d}_{\text{p}}[i] = \mathbf{d}[\sigma(i)], \quad \forall i \in \{0, \ldots, n_c - 1\}$, where σ denotes the permutation mapping generated by the $\mathcal{P}$.

(2) **GKR-based Difference Check:** $\mathcal{P}$ and $\mathcal{V}$ execute a GKR protocol to verify the correct generation of a witness vector $\mathbf{l}$, which captures the relative order of elements. The vector $\mathbf{l}$ is defined as:

$$l[i] = \begin{cases} \mathbf{d}_{\text{p}}[i+1] - \mathbf{d}_{\text{p}}[i], & \text{for } i \in \{0, \ldots, k-2\} \\ \mathbf{d}_{\text{p}}[i+1] - \mathbf{d}_{\text{p}}[k-1], & \text{for } i \in \{k-1, \ldots, k_c - 2\} \end{cases}$$

This step ensures that the differences between adjacent elements (within the top-k) and differences relative to the boundary (for the remaining elements) are correctly computed.

(3) **Range Check:** Finally, $\mathcal{P}$ and $\mathcal{V}$ leverage the Lookup protocol to perform a range check, ensuring that all elements in $\mathbf{l}$ are non-negative. This proof confirms both the ascending order of the first k elements and the boundary condition for the remaining cluster distances.

Through this multi-stage verification, $\mathcal{P}$ demonstrates that the first k elements are locally sorted and that all remaining elements in the database are bounded by the k-th smallest distance. This satisfies the following set of conditions:

$$\mathbf{d}_{\text{p}}[0] \le \mathbf{d}_{\text{p}}[1] \le \cdots \le \mathbf{d}_{\text{p}}[k-1] \quad \text{and} \quad \min_{j \ge k}(\mathbf{d}_{\text{p}}[j]) \ge \mathbf{d}_{\text{p}}[k-1]. \tag{2}$$

5.4 Non-uniform Dense Packing

Conventional ZK-ML frameworks, such as [33] or [42], typically employ a uniform padding strategy that pads all input vectors to the maximum dimension among them. In our context, this would lead to a "sparse input" problem, where the input layer is dominated by redundant zero-elements. For instance, in [33], the input layer comprises four primary vectors: the input embedding X, model parameters W, intermediate values aux, and lookup queries L. However, because the size of W often exceeds that of the other vectors by more than an order of magnitude, uniform padding would introduce computational redundancy. Analogously, this imbalance is even more pronounced in the context of VeriRAG. Specifically, in Phase I (coarse cluster filtering), the coarse centroids $\mathbf{V}_1$ are 30× larger than the other input vectors. Similarly, in Phase II (PQ-based fine-grained ranking), The length of the sub-centroid $\mathbf{V}_2$ exceeds that of other vectors by a factor of 16×.

To address the computational overhead caused by uniform padding, we introduce Non-uniform Dense Packing. Rather than aligning all input vectors to a global maximum, this strategy pads each individual witness vector to its own respective next power of two. This

approach significantly compacts the input layer; specifically, the input layer length is reduced by 4× in Phase I and 2× in Phase II. The shortened input length directly optimizes verifier time. Under the Hyrax [51] commitment scheme, the latency of the Open algorithm is dominated by the computation of $\tilde{eq}(r, b)$ for each $b \in \{0,1\}^{\ell}$, where the input layer length is 2^{ℓ} and $r \in \mathbb{F}^{\ell}$. This process requires $O(2^{\ell})$ time and is independent of the sparsity of the input layer. By employing Non-uniform Dense Packing, the verifier time is reduced by 3.2× and 1.8× in Phase I and Phase II, respectively.

6 Optimization of Retrieval Granularity via Vector-Lookup

We begin by analyzing how chunk size affects the computational overhead of Phase III verification. Next, we examine how this overhead is influenced by the vector lookup property. Finally, we show how different chunk-merging strategies enable these two aspects to synergize, achieving lower prover time without compromising retrieval quality.

6.1 Baseline Settings and Overhead Analysis

The effectiveness of a RAG pipeline is determined by the granularity of the retrieved knowledge, which is parameterized by the chunk size. In standard plaintext implementations, a chunk size of 256 tokens is widely accepted as the de facto standard [30, 46, 59]. On the one hand, excessive chunk lengths often lead to "information dilution" because using the same embedding model to compress longer textual passages into fixed-dimensional vectors can result in the loss of fine-grained semantic details. Conversely, insufficient chunk lengths fragment the semantic continuity of the source text, severing necessary contextual dependencies. Consequently, we choose 256 tokens as the baseline chunk size.

Next, we formally define the procedure for encoding chunks of tokens using calar field element and retrieving chunks with specific indexes. The whole process is illustrated in the figure 5. First, Consider a standard text chunk $\mathbf{c}$ consisting of 256 tokens. Given the standard SBERT[45] vocabulary size of $|V| = 30522$, the theoretical minimum bit-width required to represent a single token is $\lfloor \log_2 30{,}522 \rfloor = 15$ bits. To ensure system compatibility and to simplify bitwise operations, we adopt a uniform encoding scheme that allocates 16 bits (2 bytes) to each token identifier.

We then employ the widely adopted BN254 elliptic curve for cryptographic commitments. Since the scalar field of BN254 supports approximately 254 bits, it can accommodate $\lfloor 254/16 \rfloor = 15$ token identifiers within a single scalar field element. The whole process includes 2 steps: ❶ Chunk Encoding: Committing a single 256-token chunk requires partitioning the encoded data into $\lceil 256/15 \rceil = 18$ scalar field elements. ❷ Chunk Retrieval: After encoding, consider a retrieval instance in which the phase I identifies m candidate clusters $(C_{\sigma[0]}, C_{\sigma[1]}, \ldots, C_{\sigma[m-1]})$, each containing s_c chunks. To verify the Top-k Chunk Retrieval, the system performs table lookups across $w = 18$ distinct tables $\{T_1, T_2, \ldots, T_w\}$, where the size of each T_i is $m * s_c$, corresponding to the i-th scalar component of the chunks in the candidate clusters.

For a retrieved chunk c, let the witness tuple be $c = (c_1, c_2, \ldots, c_w)$, where each c_i denotes the value of the i-th scalar field element. The verification objective is to prove that for a given index $idx \in [0, |T_i| - 1]$, the following relationship holds:

$$\forall i \in [1, w], \quad c_i = T_i[idx].$$

A naive instantiation of lookup arguments would require executing w independent lookup protocols, thereby incurring a proof generation cost that grows linearly with the chunk decomposition width.

6.2 Vector Lookup and Retrieval Optimization

In this section, we first show that the vector lookup property can be directly leveraged in the aforementioned table lookup procedure to substantially reduce prover time. We then demonstrate that, enabled by vector lookups, the retrieval strategy can be optimized, leading to further performance improvements.

As analyzed above, directly verifying w independent lookup relations incurs a linear overhead in the number of tables. To eliminate this, we adopt the vector lookup primitive (Section 4.4.2), which aggregates multiple membership claims into a single proof via random linear combinations. The key observation enabling this is that although chunks are split to fit within the field, their corresponding witness values f_i share a single lookup index idx across all tables T_i. By leveraging this common index, the proof overhead is reduced to a constant, remaining independent of the chunk size.

The "constant-cost" property of vector lookup mentioned above is the fundamental enabler for our co-design strategy: it allows us to increase the retrieval granularity, i.e. the chunk size, without introducing extra cost. By reducing the number of chunks in dataset, this strategy reduces computational costs across all three phases. In Phase I, increased chunk size leads to fewer chunks and clusters, thereby minimizing the scale of the distance computation circuit. In Phase II, the ranking circuit size is also correspondingly reduced. Finally, in Phase III, the verification cost decreases due to the lower retrieval count, while the increased chunk size does not incur any additional overhead.

To prevent the lost-in-the-middle phenomenon [31] caused by excessively long prompts, we maintain a constant total token budget for the LLM. Consequently, as we increase the chunk size, we proportionally reduce the retrieval count (K). This smaller K yields a significant secondary benefit: it directly decreases the verification cost in Phase III. Plaintext experiments confirm that under this constant-budget strategy, the RAG system performance remains stable for chunk sizes up to 2048 tokens.

6.3 Chunk Merging Strategies

In this section, we introduce two chunk merging methods designed to consolidate fragmented baseline chunks (e.g., 256 tokens) into unified, larger chunks (e.g., 512 or 1024 tokens). To ensure that this aggregation minimally degrades the LLM's final generation quality, a fundamental requirement is that the merged components must be highly similar; specifically, the distance between their embedding vectors must be minimized.

The rationale behind our design is empirically demonstrated in Figure 6, which plots the embedding distances between a random target chunk and all other database chunks with respect to their indices. We observe two distinct scenarios where this semantic distance is exceptionally small. First, chunks that are adjacent in the

Protocol 1 (VeriRAG). Let $\mathbf{V}_1$ denote the coarse centroids in the inverted index, and $\mathbf{V}_2$ represent the sub-centroids (or codebooks) used for Product Quantization. For the i-th cluster ($i \in \{1, \ldots, n_c\}$), we define $\mathbf{I}_i$ as the quantized codes (or encoded indices) and $\mathbf{C}_i$ as the corresponding data chunks. The vector $\mathbf{q}$ represents the query submitted by the client.

(1) **VeriRAG.KeyGen(1^λ):** *Run* $pp \leftarrow \mathsf{zkPC.KeyGen}(1^\lambda)$ *and output* pp.

(2) **VeriRAG.Commit($\mathbf{V}_1, \mathbf{V}_2, \{\mathbf{I}_i\}, \{\mathbf{C}_i\}, pp, \mathbf{r}$):** *Define* $\tilde{\mathbf{V}}_1, \tilde{\mathbf{V}}_2, \tilde{\mathbf{I}}_i$, *and* $\tilde{\mathbf{C}}_i$ *as the MLEs of* $\mathbf{V}_1, \mathbf{V}_2, \mathbf{I}_i$, *and* $\mathbf{C}_i$. $\mathcal{P}$ *computes* $com_{\mathbf{V}_1} \leftarrow \mathsf{zkPC.Commit}(\tilde{\mathbf{V}}_1, pp, r_{\mathbf{V}_1})$ *and* $com_{\mathbf{V}_2}$. *Similarly, for each cluster* $i \in \{1, \ldots, n_c\}$, $\mathcal{P}$ *generates commitments* $com_{\mathbf{I}_i}$ *and* $com_{\mathbf{C}_i}$. *Finally,* $\mathcal{P}$ *sends the commitment set* $com_{db} = (com_{\mathbf{V}_1}, com_{\mathbf{V}_2}, \{com_{\mathbf{I}_i}\}_{i=1}^{n_c}, \{com_{\mathbf{C}_i}\}_{i=1}^{n_c})$ *to* $\mathcal{V}$. *Here, we define the composite database witness* $\mathsf{w}_{db} = (\mathbf{V}_1, \mathbf{V}_2, \{\mathbf{I}_i\}_{i=1}^{n_c}, \{\mathbf{C}_i\}_{i=1}^{n_c})$ *and its associated combined randomness* $r_{\mathsf{w}_{db}} = (r_{\mathbf{V}_1}, r_{\mathbf{V}_2}, \{r_{\mathbf{I}_i}\}_{i=1}^{n_c}, \{r_{\mathbf{C}_i}\}_{i=1}^{n_c})$.

(3) $\langle$ **VeriRAG.Prove(w_{db}, $r_{\mathsf{w}_{db}}$), VeriRAG.Verify(com_{db})** $\rangle$**($\mathbf{q}$, pp)**:

- ***Coarse Cluster Filtering:***
  - *Upon receiving the query* $\mathbf{q}$, $\mathcal{P}$ *computes the Euclidean distances* $\mathbf{d}$ *between* $\mathbf{q}$ *and the coarse centroids* $\mathbf{V}_1$. *It then identifies the top-m nearest clusters and generates the corresponding residuals* $\mathbf{r}$ *and lookup queries* $\mathbf{l}$ *required for verifiable selection.*
  - $\mathcal{P}$ *generates commitments for the input query, the distance vectors (original* $\mathbf{d}$ *and permuted* $\mathbf{d}_\mathrm{p}$*), the lookup witnesses* $\mathbf{l}$, *and the residuals* $\mathbf{r}$. *The set of commitments* $C = \{com_{\mathbf{q}}, com_{\mathbf{d}}, com_{\mathbf{d}_\mathrm{p}}, com_{\mathbf{l}}, com_{\mathbf{r}}\}$ *and the permutation* σ *are then sent to* $\mathcal{V}$. $\mathcal{P}$ *constructs a input vector* $\mathbf{in}$ *by concatenating the padded versions of* $\mathbf{V}_1, \mathbf{q}, \mathbf{l}, \mathbf{d}, \mathbf{d}_\mathrm{p}$, *and* $\mathbf{r}$.
  - *To verify the computation of distances, We leverage the property that* $\mathbf{q}$ *and* $\mathbf{V}_1$ *are pre-normalized to efficiently compute distance.* $\mathcal{P}$ *and* $\mathcal{V}$ *execute a GKR protocol to prove the correctness of* $\mathbf{d}$.
  - *To verify the selection of the top-m closest clusters,* $\mathcal{P}$ *and* $\mathcal{V}$ *execute the proposed top-m verification protocol, which comprises a permutation check, the GKR protocol, and range checks.*
  - *To verify the residual vector* $\mathbf{r}$, $\mathcal{P}$ *and* $\mathcal{V}$ *execute the GKR protocol to ensure the correctness of* $\mathbf{r}$*'s generation from the query vector and the selected cluster centroids.*
  - *At the input layer,* $\mathcal{V}$ *has received the evaluations.* $\mathcal{P}$ *and* $\mathcal{V}$ *execute a sumcheck protocol to combine them into a single evaluation* $\tilde{\mathbf{in}}(r)$. $\mathcal{V}$ *validates the evaluation of the aggregate input polynomial* $\tilde{\mathbf{in}}(r)$ *by opening the polynomial commitments.*
- ***PQ-based Fine-grained Ranking:***
  - *Within the identified m clusters,* $\mathcal{P}$ *selects the top-k nearest vectors using PQ. Specifically,* $\mathcal{P}$ *computes the distance lookup tables* $\mathbf{T}$ *(between residuals* $\mathbf{r}$ *and sub-centroids* $\mathbf{V}_2$*), retrieves the quantized sub-distances* $\mathbf{d}_{\mathrm{codes}}$, *and aggregates them into the distance vector* $\mathbf{d}'$. *Subsequently,* $\mathcal{P}$ *generates the permuted vector* $\mathbf{d}'_\mathrm{p}$ *and the lookup queries* $\mathbf{l}'$ *required for top-k verification.*
  - $\mathcal{P}$ *generates commitments:* $C' = \{com_{\mathbf{T}}, com_{\mathbf{d}_{\mathrm{codes}}}, com_{\mathbf{d}'}, com_{\mathbf{d}'_\mathrm{p}}, com_{\mathbf{l}'}\}$. *These commitments, along with the public permutation mapping* σ', *are transmitted to* $\mathcal{V}$. *Following the same procedure,* $\mathcal{P}$ *constructs a input vector* $\mathbf{in}'$ *through the concatenation of the padded second-stage components:* $\mathbf{V}_2, \mathbf{d}_{\mathrm{codes}}, \mathbf{r}, \mathbf{d}', \mathbf{d}'_\mathrm{p}$, *and* $\mathbf{l}'$.
  - *To verify the PQ-based distance computation,* $\mathcal{P}$ *and* $\mathcal{V}$ *first execute a GKR protocol to prove the correct construction of lookup tables* $\mathbf{T}$. *Subsequently, leveraging the PQ codes* $\{\mathbf{I}_{\sigma[i]}\}_{i=1}^{m}$, *they employ the Lookup protocol to certify the retrieval of sub-distances* $\mathbf{d}_{\mathrm{codes}}$ *from* $\mathbf{T}$. *Finally, another GKR layer verifies that each element in* $\mathbf{d}'$ *is correctly aggregated from its constituent sub-distances.*
  - $\mathcal{P}$ *and* $\mathcal{V}$ *verify the selection of the top-k chunks following the same procedure.*
  - *At the input layer,* $\mathcal{P}$ *and* $\mathcal{V}$ *run a sumcheck protocol to combine the values into a single evaluation* $\tilde{\mathbf{in}}'(r')$. *Finally,* $\mathcal{V}$ *validates* $\tilde{\mathbf{in}}'(r')$ *by opening the polynomial commitments.*
- ***Top-k Chunk Retrieval:***
  - $\mathcal{P}$ *constructs the table* $\mathbf{T}_c = \mathbf{C}_{\sigma[0]} \| \mathbf{C}_{\sigma[1]} \| \ldots \| \mathbf{C}_{\sigma[m-1]}$. *Using the indices* $\sigma'[0], \ldots, \sigma'[k-1]$, $\mathcal{P}$ *retrieves the target chunk vector* $\mathbf{c}$. $\mathcal{P}$ *generates* $com_{\mathbf{c}}$ *and sends it to the verifier* $\mathcal{V}$.
  - $\mathcal{P}$ *and* $\mathcal{V}$ *execute the Lookup Protocol to prove that* $\mathbf{c}[i] = \mathbf{T}_c[\sigma'[i]]$ *for all* $i \in \{0, \ldots, k-1\}$.

(4) *If the check and all the verifications of the polynomial commitments and sumchecks pass,* $\mathcal{V}$ *outputs* 1; *otherwise,* $\mathcal{V}$ *outputs* 0.

Figure 4: The VeriRAG Protocol implementation details.

source text (i.e., consecutive indices) naturally share strong narrative continuity, leading to minimal distances. Second, semantically equivalent information often appears across disparate contexts, meaning chunks with widely separated indices can also exhibit high vector proximity. Driven by these two observations, we propose two targeted merging approaches, each optimizing for a different dimension of information coherence: *Adjacent Merging*, which exploits local textual continuity, and *Semantic Neighbor Merging*, which leverages global proximity within the embedding space. The

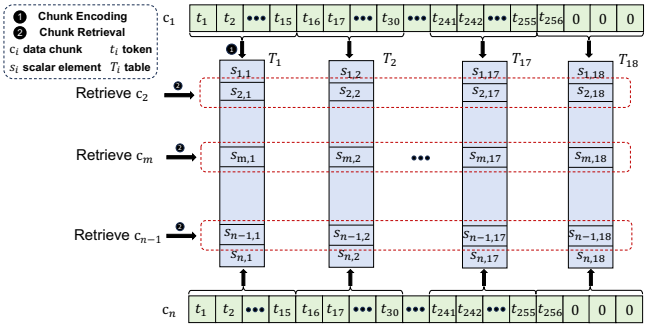

Figure 5: Process of encoding and retrieving data chunks using the same index to multiple tables.

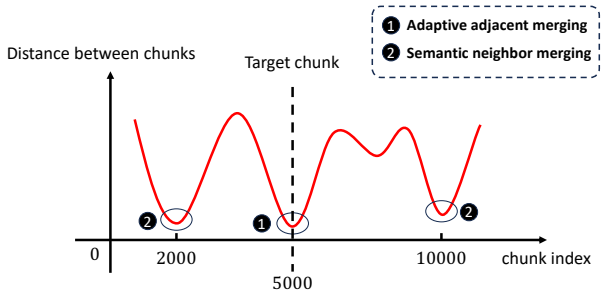

Figure 6: Distribution of embedding distances between a random target chunk and all other database chunks relative to their indices.

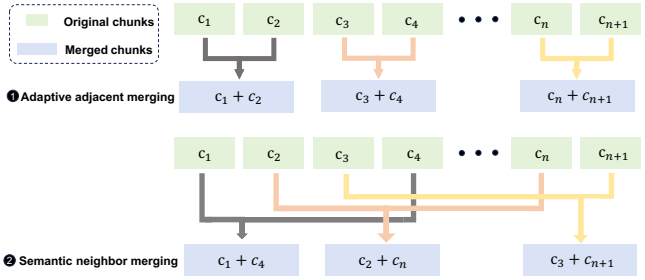

Figure 7: Illustration of adaptive adjacent merging and semantic neighbor merging strategies.

mechanics of these two aggregation approaches are illustrated in Figure 7, and their empirical impact on the downstream LLM generation quality will be evaluated in our subsequent experiments.

6.3.1 Adaptive Adjacent Merging. The first strategy prioritizes the narrative continuity of the source contexts. This strategy draws inspiration from a standard merging strategy that simply concatenates contiguous chunks (e.g., merging c_i and c_{i+1}), but solves the problem of possibly bridging unrelated contexts if c_i represents the conclusion of a semantic unit (e.g., the end of a chapter or section).

Mechanism: We introduce a Semantic Boundary Detection mechanism. Before merging a target chunk c_i with its successor c_{i+1}, we calculate the L_2 distance between their respective embedding vectors. We define a divergence threshold, τ. A "semantic drop" or topic shift likely appears if $L_2(c_i, c_{i+1}) > \tau$. Consequently, this merge is stopped, and c_{i+1} initiates a new retrieval block. This threshold-based segmentation ensures that the consolidated chunks remain semantically homogeneous. This optimization prevents noise by avoiding the concatenation of irrelevant contexts.

6.3.2 Semantic Neighbor Merging. The second strategy, *Semantic Neighbor Merging*, prioritizes global semantic proximity by aligning the offline data organization with the online retrieval criterion (specifically, the L_2 distance). The underlying intuition is straightforward: if a query vector q is highly relevant to a chunk c_i, and c_i is tightly clustered with c_j in the embedding space, then q is statistically highly probable to be relevant to c_j as well. This cross-context aggregation is strongly justified by our empirical analysis; for a randomly selected chunk, an average of 2.13 of its top-4 nearest semantic neighbors originate from its own context, indicating that nearly half of its strongest semantic matches are scattered elsewhere in the database. By grouping these scattered but semantically linked chunks, this strategy optimizes the merged blocks specifically for vector-based retrieval.

Mechanism: For a given chunk, we first employ a brute-force method to compute and store the L_2 distances between it and all chunks remaining in the database. Then we merge it with its nearest chunks before we remove them from the old database to avoid them from being merged again. This de-duplication operation ensures that the number of chunks is reduced by half, thereby leading to significant improvements in latency.

In practice, we first calculated the frequency of each chunk, which is defined as how many times this chunk is selected as the nearest neighbor by other chunks. Higher frequency means After we obtained the frequencies of all of the chunks, we prioritized merging the chunks with the highest selection counts to ensure the system's accuracy and robustness.

Limitations: A primary limitation of our method is the potential for false positive retrievals. Specifically, chunks that are close in the embedding space can sometimes be mutually confusing. For example, if two chunks describe the premium memberships for different websites, a query targeting one website might inadvertently retrieve the other as context. Such distracting information can degrade the performance and accuracy of RAG systems [31, 58], highlighting a fundamental trade-off between retrieval accuracy and system efficiency.

Operational Feasibility: While the exhaustive calculation of global pairwise distances (an $O(N^2)$ complexity) appears computationally intensive, it is strictly an offline iandexing operation. This cost is incurred only once during the system initialization phase. Therefore, it is amortized over infinite online queries and imposes zero latency penalty on the online execution of the VeriRAG pipeline.

Scalability Optimization: To further accommodate massive-scale databases where $O(N^2)$ pre-processing is prohibitive, we provide an optional heuristic optimization. Similar to the hierarchical search logic used in online retrieval, we can approximate the nearest neighbors by first identifying the top-m closest centroids to the target chunk c_i, and subsequently restricting the pairwise distance

calculation exclusively to the chunks within these selected clusters. This reduces the search space significantly while maintaining high merging precision. Experiments have shown that this cluster-based searching can get an average recall of higher than 96%.

7 Implementations

7.1 Cryptographic Implementations

We implemented the VeriRAG framework in C++, building upon the open-source codebase of zkGPT [33, 42]. Following the zkGPT implementation, we adopt the Hyrax polynomial commitment scheme [51], which offers an $O(N)$ prover complexity alongside $O(\sqrt{N})$ proof size and verification time for a polynomial of size N. We implement operations on finite field and the BN254 elliptic curve using the `mcl` library. This choice of curve aligns with [4, 32, 42] and provides a security level of approximately 100 bits.

Our experimental evaluations were conducted on a server equipped with an Intel Xeon Gold 5220R CPU (2.20GHz, 24 cores) and 376GB of DDR4 RAM. To achieve high performance, we employ 24-thread parallelization across the commitment, proving, and verification phases by default. Although the Pippenger algorithm is widely used to speed up commitments, it is suboptimal for short vectors due to its initialization costs. Consequently, in our VeriRAG framework, we perform direct commitment for the Lookup argument phase, as it typically handles shorter vectors. While, for the GKR phase, we leverage the Pippenger algorithm to maximize the throughput of commitment computations.

7.2 Benchmarks and Model Configurations

We conduct zero knowledge benchmark and breakdown across four widely adopted QA datasets[1, 21, 59] to ensure coverage of diverse retrieval and reasoning scenarios. Specifically, we use TriviaQA[25] train(2.13GB) and validation(0.25GB) splits, which we will refer to as TQA-Train and TQA-Val in the following sections; SQuAD v1.1[44](35.14MB); and KILT[40](37.32GB), which provides long-form, Wikipedia-grounded contexts.

For dense retrieval, all documents and queries are embedded using BGE-M3[7] model. We use Qwen2-7B QA model as the generation model. We adopt the F1 score and the EM(exact match) as the performance metric, as they are widely adopted in RAG systems[21, 46, 59]. All of the F1 score and EM in the following experiments are averaged over 300 samples in the corresponding datasets. The hyperparameters of K-means clustering were selected based on the guidelines provided in the Faiss[10] documentation, which suggests that, given the total number of chunks N, the number of clusters should be in range $[4\sqrt{N}, 16\sqrt{N}]$ and average cluster size in range $[30, 256]$.

8 Experiment

In this section, we first evaluate the total prover and verifier time to demonstrate VeriRAG's scalability across datasets of varying sizes. Subsequently, we provide a detailed breakdown of the execution time for each phase. Finally, in order to demonstrate the rationale behind two merging schemes, we conduct experiments to access the RAG system performance across different chunk sizes using these schemes.

Table 1: Prover time, verifier time (s) and proof size (MB) evaluation under different datasets and chunk size configurations.

Datasets	Chunk Size	Prover time	Verifier time	Proof size
SQuDA	64	11.54	0.62	0.28
	128	7.24	0.49	0.20
	256	6.57	0.46	0.26
TQA-Val	256	38.65	0.93	1.61
	512	25.87	0.64	1.27
	1024	11.94	0.51	1.23
TQA-Train	256	55.99	1.20	2.67
	512	45.72	1.01	2.54
	1024	24.07	0.69	2.34
KILT	64	264.34	2.77	4.21
	128	168.93	2.11	5.99
	256	96.18	1.61	5.21

Table 2: Comparison with V3DB. The tuple parameters (dataset size, vector dimensionality, number of PQ sub-quantizers, sub-quantizer codebook size, coarse centroids, probed clusters, retrieved vectors) are configured as (8192, 128, 8, 16, 256, 16, 64) and (65536, 256, 16, 256, 512, 64, 128).

Config	Framework	Prove (s)	Verify (s)	Proof Size (KB)	Memory (GB)
Basic	V3DB	0.78	0.007	144.20	11.08
	VeriRAG	0.45	0.16	110.73	1.98
Large	V3DB	24.16	0.011	248.99	447.23
	VeriRAG	5.49	0.39	538.69	2.05

8.1 Overall Performance of VeriRAG

Initially, we evaluate the performance of VeriRAG across four diverse benchmarks: SQuAD, TriviaQA (Validation and Training), and KILT. The datasets represent diverse data volumes, covering scales from 3 million tokens (35 MB) to as large as 700 million tokens (37 GB). To ensure consistency across trials, we maintain a fixed selection ratio for the coarse cluster filter, whereby the number of selected chunks remains a constant percentage of the total corpus. The results in Table 1 show that while the prover time scales with the dataset size (from 6.6s to 96.2s), the verifier time remains extremely efficient, staying under 1.6s even for the largest dataset. This disparity highlights VeriRAG's efficiency, as the verification overhead grows at a significantly slower rate than the data volume.

Furthermore, VeriRAG enables retrieval verification over gigabyte-scale datasets while maintaining a sub-megabyte proof size. The proof size is primarily dominated by the commitments of encoded indices and data chunks for the selected clusters, denoted as $\{\mathbf{I}_{c_{\sigma[i]}}\}_{i=1}^{m}$ and $\{\mathbf{C}_{c_{\sigma[i]}}\}_{i=1}^{m}$, respectively. The total proof size is governed by two hyperparameters: the number of coarse clusters (n_c) and the cluster size (s_c). Since the Hyrax commitment size scales as $O(\sqrt{s_c})$, the total commitment overhead is proportional to $n_c\sqrt{s_c}$. Therefore, configuring fewer clusters with larger sizes effectively minimizes the overall proof size, introducing a fundamental trade-off between proof compactness and retrieval quality.

Furthermore, we compare our system with V3DB [41], which implements its verification using Plonk and FRI. As Table 2 shows,

Table 3: Prover and Verifier Time (s) across different phases, datasets, and chunk sizes. V-Time represents the verifier time.

Datasets	Chunk	Phase I				Phase II				Phase III	
		Commit Advice	GKR	Lookup	V-time	Commit Advice	GKR	Lookup	V-time	Lookup	V-time
SQuDA	64	0.05	0.81	0.10	0.22	0.16	9.43	0.93	0.37	0.06	0.03
	128	0.06	0.60	0.08	0.18	0.12	5.90	0.48	0.31	0.07	0.02
	256	0.05	0.60	0.07	0.21	0.10	5.30	0.45	0.25	0.03	0.02
TQA-Val	256	0.08	2.45	0.19	0.30	0.29	31.99	3.65	0.63	0.07	0.03
	512	0.06	1.70	0.14	0.28	0.25	21.97	1.75	0.36	0.06	0.03
	1024	0.05	0.71	0.09	0.19	0.15	9.78	1.16	0.32	0.06	0.03
TQA-Train	256	0.11	3.71	0.28	0.44	0.33	47.24	4.32	0.76	0.39	0.09
	512	0.08	2.76	0.21	0.32	0.31	38.70	3.66	0.69	0.24	0.07
	1024	0.06	1.60	0.12	0.24	0.24	20.05	2.00	0.45	0.15	0.05
KILT	64	0.31	30.38	1.12	0.87	0.69	215.12	16.72	1.65	1.56	0.30
	128	0.18	18.29	0.65	0.71	0.50	133.39	15.92	1.46	1.29	0.26
	256	0.15	11.42	0.56	0.60	0.49	73.25	10.31	1.01	0.82	0.15

Table 4: Performance results with different merging schemes on SQuAD and TQA-Val. ♦ means Adaptive Adjacent Merging, ★ means Semantic Neighbor Merging.

Dataset	SQuAD						
Chunk size (tokens)	64 (baseline)	128 ♦	128 ★	256 ♦	256 ★	512 ♦	512 ★
F1	47.64	49.59	44.09	46.85	45.41	37.64	35.65
EM	30.00	32.50	27.50	29.00	31.00	18.33	16.67

Dataset	TQA-Val						
Chunk size (tokens)	256 (baseline)	512 ♦	512 ★	1024 ♦	1024 ★	2048 ♦	2048 ★
F1	41.06	38.44	39.44	41.49	40.03	33.78	35.00
EM	25.33	24.00	25.00	27.00	25.5	20.67	22.00

VeriRAG outperforms V3DB in terms of prover time, memory footprint. This performance advantage stems from architectural choices: Plonk intrinsically demands higher computational and memory overheads during proof generation than GKR. Regarding proof size, VeriRAG yields comparable proofs in the Basic configuration and larger ones in the Large configuration. Although our Hyrax commitments generally produce smaller proofs than FRI, VeriRAG's need to generate a commitment for each cluster offsets this inherent advantage. Finally, while VeriRAG exhibits a higher verifier time, it remains well within a practical sub-second threshold (< 1s).

8.2 Breakdown Analysis

Table 3 provides a detailed breakdown of the prover and verifier times, from which we derive two key observations.

First, the primary bottleneck for the prover is the GKR protocol in Phase II, which accounts for 75% to 85% of the total execution time. This concentration of overhead stems from two factors: (1) unlike Phase I, where vectors are normalized to allow for efficient inner-product-based distance metrics, Phase II involves distance computations over unnormalized sub-vectors; (2) smaller cluster sizes can introduce higher computational overhead for PQ. Nonetheless, PQ plays a crucial role in VeriRAG by significantly reducing the proof size. Specifically, instead of committing to every high-dimensional vector within a cluster, the prover only needs to transmit commitments for 8 indices per vector. Under the Hyrax commitment scheme, this approach yields an 8-fold reduction in commitment size per cluster, striking an effective balance between computation and communication.

Table 5: Hyperparameter settings on two datasets.

Hyperparameters	**SQuAD**	**TQA-Val**
Chunk size (tokens)	64	256
# Clusters	730	2900
# Retrieved chunks	16	16
Total tokens (per query)	1024	4096

Second, increasing the chunk size yields significant performance benefits for both Phase I and Phase II. As illustrated in Tables 1 and 3, we observe a 1.7× to 3.2× speedup in prover time and a 1.3× to 1.8× reduction in verifier latency. This performance gain is primarily attributed to the chunk-merge strategy: as the chunk size increases, the total number of embedding vectors decreases. This reduction in granularity subsequently lowers the total number of clusters and the number of selected clusters required for retrieval, thereby enhancing the execution efficiency of the cryptographic protocols in Phases I and II.

8.3 Comparison of Different Merging Schemes

We assess the RAG system performance on two datasets: TQA-Val and SQuAD. For these two datasets, both baseline setting and two merging schemes are tested. The baseline hyperparameter settings of these two datasets are listed in Tables 5.

Table 4 evaluates SQuAD (64–512 tokens) and TQA-Val (256–2048 tokens) under Adaptive Adjacent (⧫) and Semantic Neighbor (★) merging schemes. Overall, performance exhibits a non-linear trend, peaking at moderate lengths before degrading sharply at extreme sizes. Notably, Adaptive Adjacent Merging outperforms the baseline at optimal sizes, achieving peak F1 scores of 49.59 on SQuAD (128 tokens) and 41.49 on TQA-Val (1024 tokens). This confirms that moderately expanding contiguous text preserves complete reasoning context. In contrast, Semantic Neighbor Merging generally underperforms its adjacent counterpart. While it groups conceptually similar chunks, it is highly susceptible to "false positive" noise—retrieving chunks that are semantically related but factually irrelevant. This explains why standard RAG implementations predominantly adopt adjacent merging, as over-expanding or semantically jumping introduces distracting context that hinders generation.

9 Conclusion

To bridge the trust gap in RAG systems between service providers and end-users, we propose VeriRAG, the first framework designed to ensure verifiability and enhance the reliability of RAG-generated outcomes. To optimize performance, we leveraged ANNS, a specialized top-k verification protocol, and non-uniform dense packing. Additionally, our joint optimization of vector lookup primitive and chunk merging significantly lowered processing overhead without compromising retrieval verification efficiency. Experiments on real-world datasets confirm that VeriRAG scales effectively to large-scale tasks, offering a practical and secure foundation for future retrieval-augmented AI systems.

References

[1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. SELF-RAG: LEARNING TO RETRIEVE, GENERATE, AND CRITIQUE THROUGH SELF-REFLECTION. (2024).

[2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.

[3] Manuel Blum, Will Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. 1994. Checking the correctness of memories. *Algorithmica* 12, 2 (1994), 225–244.

[4] G Botrel, T Piellard, YE Housni, I Kubjas, and A Tabaie. [n. d.]. Consensys/gnark: v0. 9.0, February 2023. *URL https://doi. org/10.5281/zenodo* 5819104 ([n. d.]).

[5] Benedikt Bünz and Jessica Chen. 2024. Proofs for deep thought: Accumulation for large memories and deterministic computations. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 269–301.

[6] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. Zkml: An optimizing system for ml inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 560–574.

[7] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2025. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216 [cs] doi:10.48550/arXiv.2402.03216

[8] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. 2012. Practical verified computation with streaming interactive proofs. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. 90–112.

[9] Trisha Datta, Binyi Chen, and Dan Boneh. 2025. VerITAS: Verifying image transformations at scale. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4606–4623.

[10] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]

[11] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2017. Fast approximate nearest neighbor search with the navigating spreading-out graph. *arXiv preprint arXiv:1707.00143* (2017).

[12] Ariel Gabizon and Zachary J Williamson. 2020. plookup: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive* (2020).

[13] Ariel Gabizon and Zachary J. Williamson. 2020. Plookup: A Simplified Polynomial Protocol for Lookup Tables.

[14] Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. 2015. Delegating computation: interactive proofs for muggles. *Journal of the ACM (JACM)* 62, 4 (2015), 1–64.

[15] Shafi Goldwasser, Silvio Micali, and Chales Rackoff. 2019. The knowledge complexity of interactive proof-systems. In *Providing sound foundations for cryptography: On the work of shafi goldwasser and silvio micali*. 203–225.

[16] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *International Conference on Machine Learning*. PMLR, 3887–3896.

[17] Ulrich Haböck. 2022. Multivariate lookups based on logarithmic derivatives. *Cryptology ePrint Archive* (2022).

[18] Meng Hao, Hanxiao Chen, Hongwei Li, Chenkai Weng, Yuan Zhang, Haomiao Yang, and Tianwei Zhang. 2024. Scalable zero-knowledge proofs for non-linear functions in machine learning. In *33rd USENIX Security Symposium (USENIX Security 24)*. 3819–3836.

[19] Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume*. 874–880.

[20] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.

[21] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. arXiv:2403.14403 [cs] doi:10.48550/arXiv.2403.14403

[22] Wenqi Jiang, Marco Zeller, Roger Waleffe, Torsten Hoefler, and Gustavo Alonso. 2023. Chameleon: a heterogeneous and disaggregated accelerator system for retrieval-augmented language models. *arXiv preprint arXiv:2310.09949* (2023).

[23] Wenqi Jiang, Shuai Zhang, Boran Han, Jie Wang, Bernie Wang, and Tim Kraska. 2024. Piperag: Fast retrieval-augmented generation via algorithm-system co-design. *arXiv preprint arXiv:2403.05676* (2024).

[24] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[25] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Regina Barzilay and Min-Yen Kan (Eds.). Association for Computational Linguistics, Vancouver, Canada, 1601–1611. doi:10.18653/v1/P17-1147

[26] YuHe Ke, Liyuan Jin, Kabilan Elangovan, Hairil Rizal Abdullah, Nan Liu, Alex Tiong Heng Sia, Chai Rick Soh, Joshua Yi Min Tung, Jasmine Chiat Ling Ong, and Daniel Shu Wei Ting. 2024. Development and testing of retrieval augmented generation in large language models–a case study report. *arXiv preprint arXiv:2402.01733* (2024).

[27] Assia Khan. 2023. Retrieval Augmented Generation: 5 Uses and Their Examples. Lettria.

[28] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[29] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. Zksql: Verifiable and efficient query evaluation with zero-knowledge proofs. *Proceedings of the VLDB Endowment* 16, 8 (2023).

[30] Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettlemoyer, and Scott Yih. 2024. RA-DIT: RETRIEVAL-AUGMENTED DUAL INSTRUC- TION TUNING. (2024).

[31] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172 [cs] doi:10.48550/arXiv.2307.03172

[32] Tianyi Liu, Tiancheng Xie, Jiaheng Zhang, Dawn Song, and Yupeng Zhang. 2024. Pianist: Scalable zkrollups via fully distributed zero-knowledge proofs. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1777–1793.

[33] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*.

2968–2985.
[34] Wanqi Liu, Hanchen Wang, Ying Zhang, Wei Wang, Lu Qin, and Xuemin Lin. 2021. EI-LSH: An early-termination driven I/O efficient incremental c-approximate nearest neighbor search. *The VLDB Journal* 30, 2 (2021), 215–235.
[35] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. 1992. Algebraic methods for interactive proof systems. *Journal of the ACM (JACM)* 39, 4 (1992), 859–868.
[36] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
[37] Yusuke Matsui, Yusuke Uchida, Hervé Jégou, and Shin'ichi Satoh. 2018. A survey of product quantization. *ITE Transactions on Media Technology and Applications* 6, 1 (2018), 2–10.
[38] Shahar Papini and Ulrich Haböck. 2023. Improving logarithmic derivative lookups using GKR. *Cryptology eprint Archive* (2023).
[39] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient approximate nearest neighbor search in multi-dimensional databases. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.
[40] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. 2021. KILT: A Benchmark for Knowledge Intensive Language Tasks. arXiv:2009.02252 [cs] doi:10.48550/arXiv.2009.02252
[41] Zipeng Qiu, Wenjie Qu, Jiaheng Zhang, and Binhang Yuan. 2026. V3DB: Audit-on-Demand Zero-Knowledge Proofs for Verifiable Vector Search over Committed Snapshots. *arXiv preprint arXiv:2603.03065* (2026).
[42] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. 2025. zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference. In *34st USENIX Security Symposium (USENIX Security 25)*.
[43] Irina Radeva, Ivan Popchev, Lyubka Doukovska, and Miroslava Dimitrova. 2024. Web application for retrieval-augmented generation: Implementation and testing. *Electronics* 13, 7 (2024), 1361.
[44] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, Jian Su, Kevin Duh, and Xavier Carreras (Eds.). Association for Computational Linguistics, Austin, Texas, 2383–2392. doi:10.18653/v1/D16-1264
[45] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 3982–3992. doi:10.18653/v1/D19-1410
[46] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. arXiv:2401.18059 [cs] doi:10.48550/arXiv.2401.18059
[47] Lev Soukhanov. 2025. Logup*: faster, cheaper logup argument for small-table indexed lookups. *Cryptology ePrint Archive* (2025).
[48] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 4405–4419.
[49] Yifang Sun, Wei Wang, Jianbin Qin, Ying Zhang, and Xuemin Lin. 2014. SRS: solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index. *Proceedings of the VLDB Endowment* (2014).
[50] Kento Tatsuno, Daisuke Miyashita, Taiga Ikeda, Kiyoshi Ishiyama, Kazunari Sumiyoshi, and Jun Deguchi. 2024. AiSAQ: All-in-Storage ANNS with Product Quantization for DRAM-free Information Retrieval. *arXiv preprint arXiv:2404.06004* (2024).
[51] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. 2018. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 926–943.
[52] Haodi Wang, Yu Guo, Rongfang Bie, and Xiaohua Jia. 2023. Verifiable arbitrary queries with zero knowledge confidentiality in decentralized storage. *IEEE Transactions on Information Forensics and Security* 19 (2023), 1071–1085.
[53] Jingdong Wang, Naiyan Wang, You Jia, Jian Li, Gang Zeng, Hongbin Zha, and Xian-Sheng Hua. 2013. Trinary-projection trees for approximate nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 36, 2 (2013), 388–403.
[54] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*. 501–518.
[55] Xiang Wu, Ruiqi Guo, Ananda Theertha Suresh, Sanjiv Kumar, Daniel N Holtmann-Rice, David Simcha, and Felix Yu. 2017. Multiscale quantization for fast similarity search. *Advances in neural information processing systems* 30 (2017).
[56] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. 2019. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Annual International Cryptology Conference*. Springer, 733–764.
[57] Guangzhi Xiong, Qiao Jin, Zhiyong Lu, and Aidong Zhang. 2024. Benchmarking retrieval-augmented generation for medicine. In *Findings of the Association for Computational Linguistics ACL 2024*. 6233–6251.
[58] Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2024. MAKING RETRIEVAL-AUGMENTED LANGUAGE MODELS ROBUST TO IRRELEVANT CONTEXT. (2024).
[59] Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, and Jiaxuan You. [n. d.]. RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs. ([n. d.]).
[60] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. 2020. Transparent polynomial delegation and its applications to zero knowledge proof. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 859–876.
[61] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. vSQL: Verifying arbitrary SQL queries over dynamic outsourced databases. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 863–880.
[62] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. A zero-knowledge version of vSQL. *Cryptology ePrint Archive* (2017).
[63] Yuxin Zheng, Qi Guo, Anthony KH Tung, and Sai Wu. 2016. Lazylsh: Approximate nearest neighbor search for multiple distance functions with a single index. In *Proceedings of the 2016 International Conference on Management of Data*. 2023–2037.

A Prompt for QA tasks

Table 6 shows the LLM prompt used for QA tasks. For RAG experiments without context, simply set context to blank.

Table 6: Prompt for Summarization

Role	Content
system	You are a helpful assistant. Answer the question strictly using only the provided context. If the answer cannot be found in the context, concisely state that the information is not available.
user	Context: {context} , question: {question}

B Zero-Knowledge Arguments

A cryptographic argument system for an NP relation $\mathcal{R}$ consists of an interactive protocol executed between a computationally bounded prover $\mathcal{P}$ and a verifier $\mathcal{V}$. Upon completion of the protocol, the verifier $\mathcal{V}$ decides whether to accept or reject the prover's claim that it possesses a valid witness w satisfying $(x, w) \in \mathcal{R}$ for a given public instance x. In this context, we specifically concentrate on *arguments of knowledge*. These protocols provide a stronger cryptographic guarantee: any prover capable of making the verifier accept must explicitly compute (and thus inherently "know") the underlying witness w. Let $\mathcal{G}$ denote the algorithm responsible for generating the public parameters pp. Assuming the relation $\mathcal{R}$ is public knowledge, the formal definition is outlined below.

Definition 3 (Zero-Knowledge Argument of Knowledge)**.** Let $\mathcal{R}$ be an NP relation. A suite of algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ constitutes a zero-knowledge argument of knowledge for $\mathcal{R}$ if it satisfies the following three fundamental properties:

- **Completeness (Correctness).** If both the prover and the verifier follow the protocol honestly, the verifier will always accept a true statement. Formally, for any public parameters $\text{pp} \leftarrow \mathcal{G}(1^\lambda)$ and any valid instance-witness pair $(x, w) \in$

$\mathcal{R}$, it holds that:

$$\Pr\left[\langle \mathcal{P}(\mathsf{pp}, w), \mathcal{V}(\mathsf{pp})\rangle(x) = 1\right] = 1$$

- **Knowledge Soundness.** It is computationally infeasible for a bounded adversarial prover to convince the verifier without knowing the witness. Formally, for any probabilistic polynomial-time (PPT) adversarial prover $\mathcal{P}^*$, there exists a PPT extractor $\mathcal{E}$ with access to the execution transcript and the internal random coins of $\mathcal{P}^*$. For $\mathsf{pp} \leftarrow \mathcal{G}(1^\lambda)$ and $\pi^* \leftarrow \mathcal{P}^*(x, \mathsf{pp})$, if $\mathcal{E}^{\mathcal{P}^*}(\mathsf{pp}, x, \pi^*)$ extracts a witness w, the probability that $\mathcal{V}$ accepts while the extracted w is invalid is negligible in the security parameter λ:

$$\Pr\left[\mathcal{V}(x, \pi^*, \mathsf{pp}) = 1 \wedge (x, w) \notin \mathcal{R}\right] \leq \mathsf{negl}(\lambda)$$

- **Zero-Knowledge.** The proof process leaks absolutely no information regarding the witness w to a potentially malicious verifier. Specifically, for any PPT adversarial verifier $\mathcal{V}^*$ with auxiliary input $z \in \{0,1\}^*$, there exists a PPT simulator $\mathcal{S}$ capable of producing an indistinguishable transcript without access to w. For any $(x, w) \in \mathcal{R}$ and $\mathsf{pp} \leftarrow \mathcal{G}(1^\lambda)$, we have:

$$\mathsf{View}_{\mathcal{V}^*}\left(\langle \mathcal{P}(\mathsf{pp}, w), \mathcal{V}^*(z, \mathsf{pp})\rangle(x)\right) \approx_c \mathcal{S}^{\mathcal{V}^*}(x, z)$$

Furthermore, the protocol $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ is classified as a **succinct** argument if the total communication complexity between $\mathcal{P}$ and $\mathcal{V}$ (i.e., the proof size) is tightly bounded by $\mathrm{poly}(\lambda, |x|, \log |w|)$.

C Zero-Knowledge Polynomial Commitment

Let $\mathbb{F}$ be a finite field, and $\mathcal{F}$ denote a family of ℓ-variate polynomials over $\mathbb{F}$ bounded by a degree parameter D. We define $W_{\ell,d}$ as the set of all monomials within this family, with a total size of $N = |W_{\ell,D}| = (D+1)^\ell$. A zero-knowledge verifiable polynomial commitment (zkPC) scheme for a polynomial $f \in \mathcal{F}$ evaluated at a point $t \in \mathbb{F}^\ell$ consists of the following probabilistic algorithms:

- $\mathsf{pp} \leftarrow \mathsf{zkPC.KeyGen}(1^\lambda)$: Takes the security parameter λ as input and generates the public parameters pp.
- $\mathsf{com} \leftarrow \mathsf{zkPC.Commit}(f, r_f, \mathsf{pp})$: Commits to the polynomial f using blinding randomness r_f, outputting a commitment string com.
- $((y, \pi), b) \leftarrow \langle \mathsf{zkPC.Open}(f, r_f), \mathsf{zkPC.Verify}(\mathsf{com})\rangle(t, \mathsf{pp})$: An interactive protocol between a prover and a verifier. The prover computes the evaluation $y = f(t)$ along with a proof π. The verifier outputs a decision bit $b \in \{0,1\}$ indicating acceptance or rejection.

Definition 4 (zkPC Properties)**.** A secure zkPC scheme must satisfy the following three fundamental properties:

- **Completeness.** An honest prover can always convince an honest verifier. For any valid polynomial $f \in \mathcal{F}$ and evaluation point $t \in \mathbb{F}^\ell$, if pp and com are generated following the protocol, the verification algorithm will output $b = 1$ with probability 1.
- **Knowledge Soundness.** It is computationally infeasible for a bounded adversary to forge an opening. Specifically, for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ that successfully opens a commitment com^*, there exists a PPT extractor $\mathcal{E}$ that can observe $\mathcal{A}$'s execution and extract a valid polynomial $f^* \in \mathcal{F}$ and randomness r_{f^*} consistent with com^*. The probability that $\mathcal{A}$ successfully convinces the verifier of an incorrect evaluation $y^* \neq f^*(t)$ is bounded by $\mathsf{negl}(\lambda)$.
- **Zero-Knowledge.** The commitment and the interactive opening process leak absolutely no information about the underlying polynomial f, other than the evaluation result $y = f(t)$. Formally, for any PPT adversary $\mathcal{A}$, the view of the real protocol execution is computationally indistinguishable from an ideal execution generated by a simulator $\mathcal{S}$, where $\mathcal{S}$ is only given oracle access to the evaluation of the polynomial.